02-620 Week 11

Machine Learning for Scientists

Aidan Jan

April 3, 2026

Hidden Markov Models (HMMs)

For example: speech processing.

- Observations: sound signals
- Hidden states: parts of speech, words, etc.

The First Use of HMMs in Biology

Finding Genes: The first draft of a human genome sequence (2001):

```
cggtgaaactgcacgattgttgctggcttaaagatagaccaatcagagtgtgtaacgtct
atttagcgtcttctatcatccaatcactgcactttacacactataaatagagcagctcag
tgggcgtatttgcgctagtgttgggtgttccgctgtgctgtttttccgtcatggctcgca
tcctaagcaaactgctcggaagtctactggtggcaaggcgccacgcaaacagttggccac
aggcagcccgcaaaagcgctccggccaccggcggcgtgaaaaagccccaccgctacccgg
gcaccgtggctctgcgcgagatccgccgttatcagaagtccactgaactgcttattcgta
aactacctttccagcgcctggtgcgcgagattgcgcaggactttaaaacagacctgcgtt
tccagagctccgctgtgatggctctgcaggaggcgtgcgaggcctacttggtagggctat
ttgaggacactaacctgtgcgccatccacgccaagcgcgtcactatcatgcccaaggaca
tccagctcgcccgccgcatccgcggagagagggcgtgattactgtggtctctctgacggt
ccaagcaaaggctcttttcagagccaccacctttcaagtaaagtagctgtaagaaacca
 atttaagacaaaagggaatgcattgggagcactttcgttttaatgctactgaaggc
```

Where are the genes?

- In this case, the observations are the nucleotides
- The hidden states are introns, exons, and intergenic regions

Researchers David Haussler and Jim Kent used HMMs to search for which base pairs were more likely to be genes:

```
cggtgaaactgcacgattgttgctggcttaaagatagaccaatcagagtgtgtaacgtct
atttagcgtcttctatcatccaatcactgcactttacacactataaatagagcagctcag
tgggcgtatttgcgctagtgttgggtgttccgctgtgctgtttttccgtcatggctcgca
tcctaagcaaactgctcggaagtctactggtggcaaggcgccacgcaaacagttggccac
aggcagcccgcaaaagcgctccggccaccggcggcgtgaaaaagccccaccgctacccgg
gcaccgtggctctgcgcgagatccgccgttatcagaagtccactgaactgcttattcgta
aactacctttccagcgcctggtgcgcgagattgcgcaggactttaaaacagacctgcgtt
tccagagctccgctgtgatggctctgcaggaggcgtgcgaggcctacttggtagggctat
ttgaggacactaacctgtgcgccatccacgccaagcgcgtcactatcatgcccaaggaca
```

```
tccagctcgcccgccgcatccgcggagagagggcgtgattactgtggtctctctgacggt
ccaagcaaaggctcttttcagagccaccacctttcaagtaaagtagctgtaagaaacca
 atttaagacaaaagggaatgcattgggagcactttcgttttaatgctactgaaggc
```

- Blue: intergenic regions
- Green: Introns
- Red: Exons

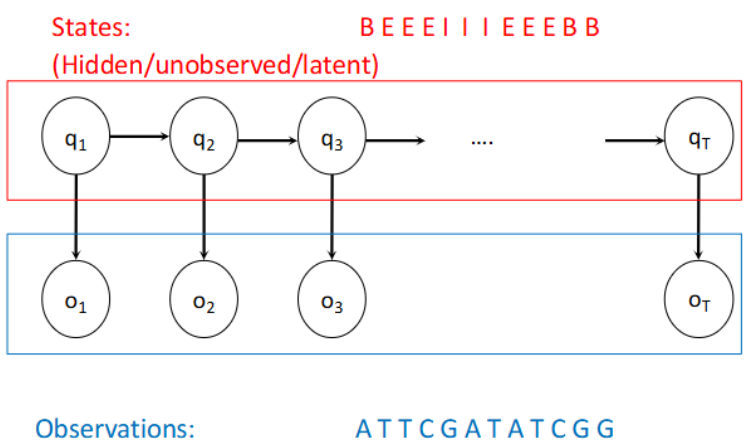
Overview

A Hidden Markov Model (HMM) is used to model a set of observations with a set of hidden states.

- Hidden states generate observations
- Hidden states transition to other hidden states

Hidden Markov Model as Bayesian Network

- The joint probability if (Q, O) is defined as



- The q_i's are "states". These are random variables that take values from a set $S = \{s_1, \ldots, s_n\}$, e.g., $\{I, E, B\}$ in the gene example.
- The o_i's are "outputs". These are random variables that take values from a set $\Sigma = \{\sigma_1, \ldots, \sigma_m\}$, e.g., $\{A, C, T, G\}$ in the gene example.

HMM definition: Conditional Probability

- We just defined random variables. How do we define the conditional probability distributions for each random variable?
  - Initial state probability
  - Transition probability
  - Emission probability

Conditional Probability Parents

q_1:

- $P(q_1 = s_i) = \pi_i$, with $\Pi = \{\pi_1, \ldots, \pi_n\}$
- The probability that we *start* at state s_i, $i = 1, \ldots, n$
- Called "initial state probability"

$q_2, \ldots, q_T$:

- $P(q_t = s_i | q_{t-1} = s_j)$
- Defined as a $n \times n$ matrix
- Called a "transition probability"

$$P(q_2|q_1) = P(q_3|q_2) = \cdots = P(q_T|q_{T-1})$$

$o_1, \ldots, o_T$:

- $P(o_t = \sigma_j | q_t = s_i)$
- Defined as a $n \times m$ matrix for n states and m output values
- Called "emission probability"

The **joint probability** of $(Q = \{q_1, \ldots, q_T\}, O = \{o_1, \ldots, o_T\})$ is defined as

$$P(Q, O) = P(q_1)P(o_1|q_1)\prod_{t=2}^{T} P(q_t|q_{t-1})P(o_t|q_t)$$

- Q represents the hidden states, and O represent the observed states
- The first term ($P(q_1)$) is the initial state probability
- The second term and the last term ($P(o_1|q_1)$ and $P(o_t|q_t)$) are emission probabilities
- $P(q_t|q_{t-1})$ are transition probabilities

HMM Summary

Random variables:

- States (latent): $q_1, \cdots, q_T$
- Outputs (observed): $o_1, \cdots, o_T$

Probability distribution

- Initial state probability: $P(q_1 = s_i)$
- Transition probability: $P(q_t = s_i | q_{t-1} = s_j)$
- Emission probability: $P(o_t = \sigma_j | q_t = s_i)$

HMM Example: Rolling one of 2 dice

- A sequence of dice rolls
- For each time, choose from two dice A and B, each with 6 faces
- Current die depends on the previous die

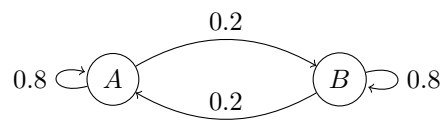
Hidden states:	A	A	A	B	B	B	B	B	B	A	A
Observations:	1	6	2	1	4	2	1	1	1	2	5

Initial State Probability:

$$\begin{cases} \Pi_A = 0.7 \\ \Pi_b = 0.3 \end{cases}$$

Emission Probability		
v	$P(v\|A)$	$P(v\|B)$
1	0.3	0.1
2	0.2	0.1
3	0.2	0.1
4	0.1	0.2
5	0.1	0.2
6	0.1	0.3

Transition Probability		
$t \setminus t+1$	A	B
A	0.8	0.2
B	0.2	0.8



HMM Inference: Forward-Backward

- Can you infer "unobserved" from "observed"? $P(Q|O)$
- In the DNA example, we would have q_t : {Exon, Intron, Background}, $o_t : \{A, T, C, G\}$
- What is the probability of candidate intron/exon/intergenic region labels given the nucleotide sequence?

The Forward-Backward Algorithm is the following:

- Compute forward variables
- Compute backward variables
- Given results of the above, perform inference
  - $P(Q|O)$ for a sequence of states $Q = \{q_1, \ldots, q_T\}$, given observations $O = \{o_1, \ldots, o_T\}$
  - $P(q_t = s_i|O)$, given observations $O = \{o_1, \ldots, o_T\}$
  - Find the most probable path (of states) $\arg\max_Q P(Q|O)$

The Forward Algorithm

- Goal: $\alpha_t(i) = P(o_1, \cdots, o_t, q_t = s_i)$, for $t = 1, \cdots, T$ and $s_i = 1, \cdots, n$
- Initialization: $\alpha_1(i) = P(o_1, q_1 = s_i)$, by definition. We can write this as $P(o_1|q_1 = s_i)P(q_1 = s_i)$.
  - The first probability is the emission probability, and the second is the initial state.
- Forward recursion: update $\alpha_{t+1}(i)$ given $\alpha_t(j), j = 1, \cdots, n$.

$$\alpha_{t+1}(i) = \sum_{j=1}^{n} P(o_{t+1}|q_{t+1} = s_i)P(q_{t+1} = s_i|q_t = s_j)\alpha_t(j)$$

This can be simplified to

$$\sum_{j=1}^{n} P(o_{t+1}|q_{t+1} = s_i)P(q_{t+1} = s_i|q_t = s_j)\alpha_t(j)$$

- The first probability in the sum is emission probability. The second is transition probability.

The Backward Algorithm

- Goal: $\beta_t(i) = P(o_{t+1}, \cdots, o_T|q_t = s_i)$, for $t = 1, \cdots, T$ and $s_i = 1, \cdots, n$
- Initialization: $\beta_T(i) = P(o_{T+1}|q_T = s_i) = 1$
- Backward recursion: update $\beta_t(i)$ given $\beta_{t+1}(j)$, $j = 1, \cdots, n$.

$$\beta_t(i) = \sum_{j=1}^{n} P(q_{t+1} = s_j|q_t = s_i)P(o_{t+1}|q_{t+1} = s_j)\beta_{t+1}(j)$$

This can be simplified to

$$\sum_{j=1}^{n} \beta_{t+1}(j)P(o_{t+1}|q_{t+1} = s_j)P(q_{t+1} = s_j|q_t = s_i)$$

- The beta term is recursion, the first probability in the sum is the transition probability, and the second is the emission probability.

HMM: Inference

- Given the results of the forward-backward algorithm, perform inference
  - $P(Q|O)$ for a sequence of states $Q = \{q_1, \ldots, q_T\}$, given observations $O = \{o_1, \ldots, o_T\}$
  - $P(q_t = s_i|O)$, given observations $O = \{o_1, \ldots, o_T\}$
  - Find the most probable path (of states) $\arg\max_Q P(Q|O)$

What is $P(Q|O)$, for a sequence of states $Q = \{q_1, \ldots, q_T\}$ given observations $O = \{o_1, \ldots, o_T\}$?

- We can use Bayes rule! We know that $P(O|Q) = P(o_1|q_1)P(o_2|q_2)\ldots P(o_T|q_T)$
- We also know $P(Q) = P(q_1)P(q_2|q_1)\ldots P(q_T|q_{T-1})$
- But how do we find $P(O)$? $P(O) = P(o_1, \ldots, o_T) = \sum_i P(o_1, \ldots, o_T, q_T = s_i) = \sum_i \alpha_T(i)$
  - For n states and a sequence of length T, finding $P(O)$ takes $O(n^2T)$ time! The others are linear in T ($O(T)$)

Can we find $P(q_t = s_i|O)$, given observations $O - \{o_1, \ldots, o_T\}$?

- Using Bayes rule, we have

$$\begin{aligned} P(q_t = A|o_1, \ldots, o_T) &= \frac{P(o_1, \ldots, o_t, o_{t+1}, \ldots, o_T, q_T = A)}{P(o_1, \ldots, o_T)} \\ &= \frac{P(o_1, \ldots, o_t, q_t = A)P(o_{t+1}, \ldots, o_T|q_t = A)}{P(o_1, \ldots, o_T)} \\ &= \frac{\alpha_t(A)\beta_t(A)}{\sum_i \alpha_t(i)\beta_t(i)} \end{aligned}$$

- We can in fact solve for $P(q_t = s_i|O)$ using the forwards and backwards results!

To find the most probable path, we solve $\arg\max_Q P(Q|O)$.

- This can be rewritten using Bayes rule to be $\frac{P(O|Q)P(Q)}{P(O)}$, which we can drop the $P(O)$ since it does not depend on Q.
- We do this using the **Viterbi algorithm**

The Viterbi Algorithm

- $\delta_t(i)$: maximum probability of ending up at state s_i at observation t (out of all possible sequences of states leading up to it)
- Initialization: $\delta_1(i) = P(o_1|q_1 = s_i)P(q_1 = s_i)$
- Update: $\delta_{t+1}(i) = \max_j P(o_{t+1}|q_{t+1} = s_i)P(q_{t+1} = s_i|q_t = s_j)\delta_t(j)$

Once we have $\delta_T(i)$, we can solve for our $P(Q^*|O)$ by:

$$P(Q^*|O) = \max_Q P(Q|O)$$

Where $Q^* = \arg\max_Q P(Q|O)$, or the path defined by $\arg\max_j \delta_t(j)$

- Goal: find most probable path $Q = [q_1, \cdots, q_T]$ given observation $O = [o_1, \cdots, o_T]$
- $\delta_t(i) = \max_{q_1,\cdots,q_t} P(q_1, \cdots, q_t|o_1, \cdots, o_t)$
- Solve by iteration:
  - $\delta_1(i) = P(o_1|q_1 = s_i)P(q_1 = s_i)$
  - $\delta_{t+1}(i) = \max_j P(o_{t+1}|q_{t+1} = s_i)P(q_{t+1} = s_i|q_t = s_j)\delta_t(j)$

Learning HMMs

The graph structure is always "known" or "fixed". Learning the parameters, or the conditional probability distributions:

- The hidden variables are observed: MLE
- The hidden variables are unobserved: EM

Learning when Hidden States are Observed

- Assume both hidden and observed variables are observed
- Data: $((O^1, Q^1), \ldots, (O^k, Q^k))$ for K sequences, where $O^k = (o_1^k, \ldots, o_T^k)$, $Q^k = (q_1^k, \ldots, q_T^k)$

MLE for learning:

$$\arg\max \log p((O^1, Q^1), \ldots, (O^k, Q^k)) = \prod_k P(O^k, Q^k)$$
$$= \arg\max sum_k \log p(q_1^k) + \sum_k \log p(o_1^k|q_1^k) + \sum_k \sum_t \log p(o_t^k|q_t^k) + \sum_k \sum_t \log p(q_t^k|q_{t-1}^k)$$

- Note that the first sum only involves initial probabilities, the second and third terms only involve emission probabilities, and the last term only involves transition probabilities
- Differentiate with respect to each of the parameters and set it to 0 to solve!
- Closed-form solution.

However, in most cases, we do not know what states generated each of the outputs (states are unobserved/hidden)

- but had we known, it would be very easy to determine an emission and transition model with MLE!
- On the other hand, if we had such a model we could determine the set of states using the inference methods from before

Expectation Maximization (EM)

- Appropriate for problems with 'missing values' for the variables.
- For example, in HMMs we usually do not observe the hidden variables (the states)
- Assume complete data log likelihood and maximize **expected log likelihood**

$$\begin{aligned}&\arg\max E[\log p((O^1, Q^1), \ldots, (O^k, Q^k))]\\ &= \arg\max E\left[\log \prod_k \left[p(q_1^k)p(o_1^k|q_1^k)\prod_{t=2}^{T} p(q_t^k|q_{t-1}^k)p(o_t^k|q_t^k)\right]\right]\end{aligned}$$

where the expectation is taken with respect to $p(Q|O,\ \text{parameters})$

Aside: EM Steps

There are two steps:

- E step: Fill in the missing variables with the expected values
- M step: Regular maximum likelihood estimation (MLE) using the values computed in the E step and the values of the other variables

This is guaranteed to converge, though only to a local minima.

E-Step

In our example, with complete data, we needed "actual counts"

- #A, #B to estimate initial probabilities and emission probabilities
- #AA, #AB, #BA, #BB to estimate transition probabilities

When hidden states are not observed, we need "expected counts" in the E step.

State and Transition Probabilities

- Probability of a state given observations: (definition)

$$P(q_t = s_i|o_1, \ldots, o_T) = \frac{\alpha_t(i)\beta_t(i)}{\sum_j \alpha_t(j)\beta_t(j)} = S_t(i)$$

- We can also derive a transition probability given observations: (definition)

$$\begin{aligned}&P(q_t = s_i, q_{t+1} = s_j|o_1, \cdots, o_T)\\ &= \frac{\alpha_t(i)P(q_{t+1} = s_j|q_t = s_i)P(o_{t+1}|q_{t+1} = s_j)\beta_{t+1}(j)}{\sum_j \alpha_t(j)\beta_t(j)} = S_t(i,j)\end{aligned}$$

First, we want to compute $S_t(i)$ and $S_t(i,j)$ for all t, i, and j $(1 \le t \le T, 1 \le i \le k, 1 \le j \le k)$

$$\begin{aligned}&P(q_t = s_i|o_1, \ldots, o_T) = S_t(i)\\ &P(q_t = s_i, q_{t+1} = s_j|o_1, \ldots, o_T) = S_t(i,j)\end{aligned}$$

M-Step

Compute transition probabilities, $a_{i,j} = p(q_t = s_j | q_{t-1} = s_i)$

$$a_{i,j} = \frac{\hat{n}(i,j)}{\sum_k \hat{n}(i,k)}$$

where

$$\hat{n}(i,j) = \sum_t S_t(i,j)$$

Now, we compute emission probabilities (assume a multinomial distribution): Let

$$B_k(j) = \sum_{t|o_t=j} S_t(k)$$

then **the emission probability** $b_k(j) = P(o_t = j | s_i = k)$

$$b_k(j) = \frac{B_k(j)}{\sum_i B_k(i)}$$

Summary EM-Algorithm

The EM algorithm for learning parameters of HMMs is the **Baum-Welch** algorithm.

Inputs: Observations $O_1, \ldots, O_T$, S = number of states in HMM

1. Guess initial transition and emission parameters
2. Compute **E-step**: $S_t(i)$ and $S_t(i,j)$
3. Compute **M-step**
4. Convergence? (If not, go to step 2)
5. Output complete model.

States in HMM

How to decide on the number of states in HMM?

- More states means a more complex model, overfitting!

Model selection problem

- Cross validation
- Nonparametric Bayesian model

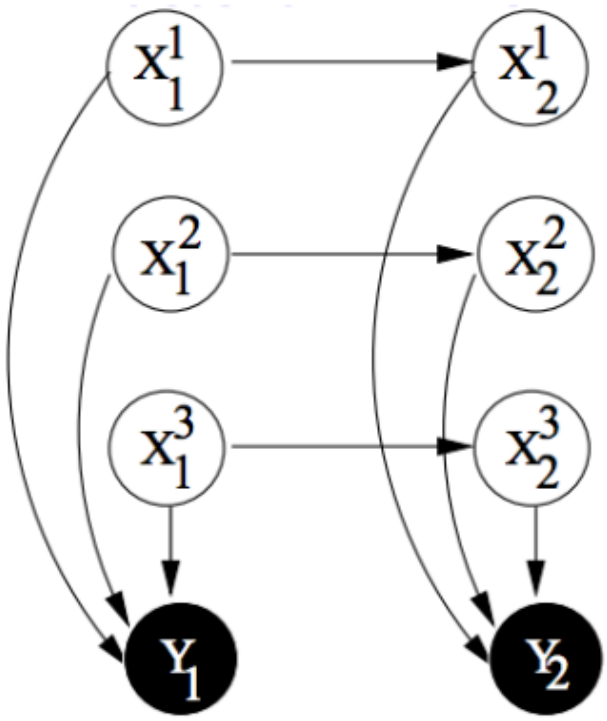
Other Forms of Bayesian Networks

Dynamic Bayesian Networks (DBNs)

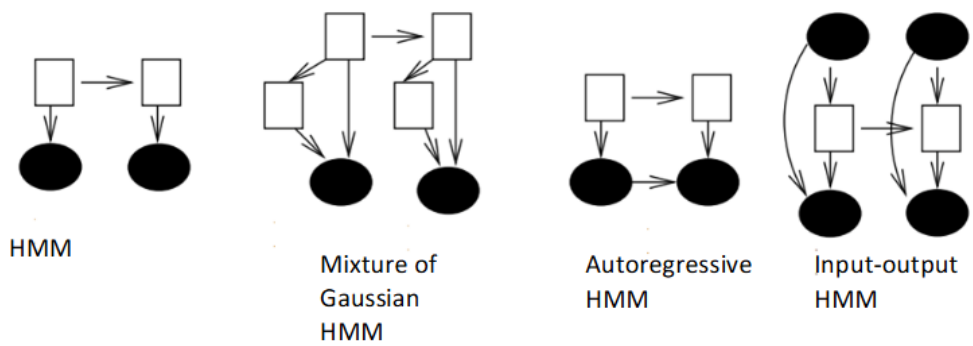
- Bayesian networks for modeling dynamic process. HMM is a special case of DBN.
  - HMM represents the state with **a single random variable**: $P(Q_t | Q_{t-1})$
  - DBN represents the state with **a set of random variables**: $P(Q_t | Q_{t-1})$, where Q_t is a set of variables

Factorial HMMs

A Factorial HMM is a DBN with D chains, each with K states.

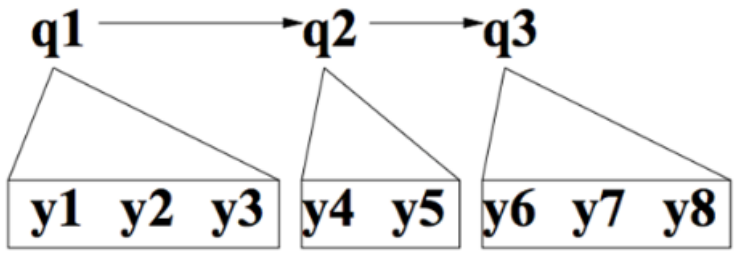


Other Variants of HMMs as DBNs



Semi-Markov HMM

- Relax the Markov constraint to allow staying in the current state for an explicit duration of time L_t

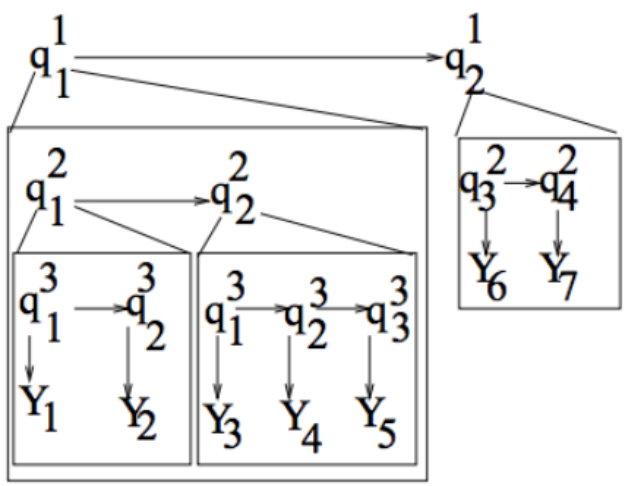


$$P(L_t = l \mid Q_t) = Poisson(\lambda_{Qt})$$

$$P(Y_{t-l+1:l} | Q_t, L_t = l) = \prod_{i=1}^{l} P(Y_i | Q_t)$$

Hierarchical HMM

- Each state can emit another HMM that generates sequences



Practical Uses of HMMs

In biology, it can be used to

- find DNA Motifs bound by transcription factor.
- sequence alignment (single and multiple species)
- Phylo-HMM (phylogenetics)

Summary: Important Points

- Why HMMs?
- Inference in HMMs
  - Forward-backward algorithm
  - Various inference tasks

  - Viterbi algorithm
- Learning in HMMs
  - EM algorithm with inference as a subroutine

Bayesian Learning

General Strategies for Learning a Probability Model

- Maximum Likelihood Estimation (MLE)
  - Frequentist approach (θ is deterministic)
  - $\hat{\theta}_{MLE} = \arg\max_\theta \log P(data|\theta)$
- Maximum *a posteriori* estimation (MAP):
  - Bayesian approach (θ is random)
  - $\hat{\theta}_{MAP} = \arg\max_\theta \log P(\theta|data)$
- Fully Bayesian appraoch
  - Estimate $P(\theta|data)$ (full distribution, not point estimate)
  - Close-form solution, direct sampling. Gibbs sampling, etc.

Example: Bayesian Inference for Linear Regression

Model: $y = Xw + e$

- Input: $X \in \mathbb{R}^{n\times m}$; output $y \in \mathbb{R}^n$
- Parameters: $w \in \mathbb{R}^m \sim N(0, \sigma_w^2)$ (IID)
- Noise: $e \in \mathbb{R}^n \sim N(0, \sigma_e^2)$ (IID)

Posterior distribution:

$$P(w|y, X) \propto P(y|X, w)P(w)$$

- $P(y|X, w) = N(Xw, \sigma_e^2)$
- $P(w) = N(0, \sigma_w^2)$

MAP estimate:

$$\hat{w}_{MAP} = \left(X^T X + \frac{\sigma_e^2}{\sigma_w^2} I\right)^{-1} X^T y$$

- Small σ_0^2 value means strong prior belief

Fully Bayesian estimation: full distribution vs. point estimate

$$P(w|y, X) = \frac{P(y|w, X)P(w)}{P(y|X)}$$

Point Estimate vs. Full Distribution

- MAP and MLE returns a point, w^* that maximizes $P(w|Data)$.
- Fully Bayesian returns a full distribution of $P(w|Data)$

Fully Bayesian Linear Regression

- How can we learn the posterior $p(w|y, X)$?
  - Both likelihood and prior are Gaussian
  - By conjugacy, the posterior is also Gaussian

$$p(w|y, X) \sim N(\mu, \Sigma)$$

  where
  - $\mu = \left(\frac{X^T X}{\sigma_e^2} + \frac{I}{\sigma_w^2}\right)^{-1} \left(\frac{X^T y}{\sigma_e^2}\right)$
  - $\Sigma = \left(\frac{X^T X}{\sigma_e^2} + \frac{I}{\sigma_w^2}\right)^{-1}$

General Bayesian Inference

Steps:

1. Set up the probability model (e.g., mixture models, HMMs, linear regression, Bayesian networks, logistic regression, etc.)
2. Determine the prior
3. Determine the posterior distributions, based on Steps (1) and (2)

Step 3: Easy case with Closed-Form Solutions

- How can you describe $p(w|Data)$
  - Parametric distribution for simple model (e.g., Gaussian)
  - Conjugacy
    - Gaussian likelihood + Gaussian prior $\rightarrow$ Gaussian posterior
    - Binomial likelihood + Beta prior $\rightarrow$ Beta posterior

Step 3: Difficult case with Numerical Solutions

- How can you describe $p(w|Data)$
  - The exact form of the posterior distribution cannot be obtained for more complex models
  - Describe the posterior with a histogram of samples from $p(w|Data)$

Posterior Inference for Linear Regression: Sampling as an Alternative Solution

- How can we learn the posterior $p(w|y, X)$?
  - How can we learn the posterior $p(w|y, X)$?
    - Both likelihood and prior are Gaussian
    - By conjugacy, the posterior is also Gaussian
  - Draw samples from this posterior $p(w|y, X) \sim N(\mu, \Sigma)$
    - Draw 100 samples $w^1, \cdots, w^{100}$ from $p(w|y, X) \sim N(\mu, \Sigma)$
  - Compute any statistics of interest from the samples.

General Numerical Approach: Sample from the Posterior Distribution

Direct sampling

- Draw 100 samples from $P(w|y, X)$
- For $i = 1, \ldots, 100$:
  - Draw w^i from $P(w|y, X)$
  - This yields $w^i = [w_1^i, \cdots, w_m^i]$

When to use it:

- Posterior doesn't have a closed-form solution, but we can sample from it.

What if:

- We cannot sample from the joint distribution $P(w|Data)$,
- but we can **sample from the conditional distribution** $P(w_j|w_{-j}, Data)$ for each coordinate? (w_{-j} represents all elements of w except w_j)

Gibbs Sampling

Compare to direct sampling, Gibbs sampling:

Draw 100 samples from $P(w|y, X)$. To do this:

- Initialize w^0 with random values
- For $i = 1, \ldots, 100$,
  - For $j = 1, \ldots, m$,
    - sample w_j^i from $p(w_j|w_{1:j-1}^i, w_{j+1:m}^{i-1}, Data)$ where $w_{1:j-1}^i$ is the 1 to $(j-1)$-th elements of w^i, and $w_{j+1:J}^{i-1}$ is the $(j+1)$ to m-th elements of w^{i-1}.

The difference compared to direct sampling is that here, we are sampling from the **marginal distribution**, which is way easier than sampling from the joint distribution as in direct sampling.

- If you run Gibbs sampling for many iterations, the samples from later iterations will approximate smaples from the target distribution, as if it were drawn from direct sampling

To draw a single sample w^i in a single iteration,

$$P(w_1, w_2, w_3, w_4, \cdots, w_m|y, X)$$

- w_1 is sampled from $P(w_1|w_{2:m}^{i-1}, y, X)$
- w_2 is sampled from $P(w_2|w_1^i, w_{3:m}^{i-1}, y, X)$
- w_3 is sampled from $P(w_3|w_{1:2}^i, w_{4:m}^{i-1}, y, X)$
- w_4 is sampled from $P(w_4|w_{1:3}^i, w_{5:m}^{i-1}, y, X)$
- $\ldots$
- w_m is sampled from $P(w_m|w_{1:m-1}^i, y, X)$

Properties of Gibbs Sampling

Initial samples are bad. But when the iteration stabilizes:

- The samples approximate the joint distribution of all variables.
- The marginal distribution of any subset of variables can be approximated by simply considering the samples for that subset of variables, ignoring the rest.
- The expected value of any variable can be approximated by averaging over all the samples.

We often discard the initial (burn-in) samples and keep the ones that are equivalent to drawing from $p(w|y, X)$

How Many "Initial" Iterations?

How many burn-in iterations?

- Too few iterations?
- Too many iterations?
- (we wait until the distribution stabilizes, and discard samples from before that point)

How many samples should we draw?

- Too few iterations?
- Too many iterations?
- (We wait until the estimates stabilize.)

Markov Chain Monte Carlo (MCMC)

- In general, in more complex models, direct sampling from posterior distribution is not possible.
- However, sampling from **a conditional posterior**, a distribution for a single variable conditional on all other variables, may be possible. Gibbs sampling!
- For even more complex models, sampling from this conditional distribution is not possible. Then, other **Markov chain Monte Carlo (MCMC)** methods!
  - Approximate the conditional posterior with a tractable distribution (Gaussian, uniform) to draw samples.

One example of a different MCMC sampling is Metropolis-Hastings sampling (sampling multiple dimensions at once)

Gibbs Sampling and MCMC in Biology

- Gibbs sampling or MCMC is widely used for Bayesian learning / inference
  - Hidden Markov models: genome phasing
  - Mixture models: clustering genes, individuals, tumors
  - Probabilistic graphical models for discovering population structure (admixture model)

Summary

- Bayesian Learning:
  - Fully Bayesian approach to learn the posterior distribution
  - Gibbs sampling as a special case of MCMC